

NEXAFORGE AI

Prompt Contract Engineering Standard

A disciplined method for designing model instructions, structured outputs, tools, and regression-safe prompt changes

Document ID	Version	Owner	Classification
NF-AI-004	2.0	LLM Application Engineering	Confidential - Internal Use

Purpose

Establish prompts as versioned software contracts rather than informal prose. The standard defines how teams specify objectives, authority boundaries, inputs, outputs, tools, error handling, safety constraints, and test coverage for every production model interaction.

Keywords: prompt engineering, contracts, structured output, schemas, tool calling, regression testing

Document Control

Field	Value
Status	Approved methodology baseline
Effective date	21 July 2026
Review cycle	Every 12 months or upon material change
Primary owner	LLM Application Engineering
Related standards	Security, privacy, software delivery, incident response, and data governance standards
Supersedes	Version 1.0 concise edition

How to Use This Document

Teams should use this document as a design and review guide, not as a substitute for engineering judgment. Sections describe mandatory controls, recommended practices, decision points, and evidence expected at release. Tailoring is permitted when documented and approved by the accountable owner. Any deviation that increases risk must include compensating controls and a review date.

1. Purpose and Objectives

Establish prompts as versioned software contracts rather than informal prose. The standard defines how teams specify objectives, authority boundaries, inputs, outputs, tools, error handling, safety constraints, and test coverage for every production model interaction.

2. Scope and Applicability

This standard applies to new implementations, major changes, vendor replacements, and material expansions of systems that rely on prompt contract engineering standard. It covers prototypes that process real organizational data, pre-production pilots, and production services. Small experiments using synthetic data may follow a reduced process, but they must still document ownership, purpose, and data handling. The methodology does not replace legal, security, privacy, or domain-specific requirements; instead, it provides the engineering structure through which those requirements are implemented and verified.

3. Core Principles

Separate policy from task: Global rules, application policy, task instructions, and user content occupy distinct layers.

Make outputs machine-verifiable: Prefer explicit schemas, enums, required fields, and validation over natural-language expectations.

Constrain freedom where failure is costly: Use narrow decision spaces for classification, routing, actions, and compliance checks.

Prompts require tests: Every change must be evaluated on representative, adversarial, and regression cases.

Design for model uncertainty: Contracts define abstention, clarification, and escalation behavior.

4. Lifecycle and Detailed Procedure

1. Contract definition

Document objective, non-objectives, caller, inputs, allowed knowledge, output consumer, and consequences of error.

- Required evidence: a reviewable artifact or trace showing that contract definition was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

2. Instruction layering

Place immutable policy first, application behavior second, task-specific context third, and untrusted user or document content last. Delimit each layer clearly.

- Required evidence: a reviewable artifact or trace showing that instruction layering was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

3. Input schema

Define required and optional fields, types, ranges, sanitization, maximum lengths, and how missing values are represented.

- Required evidence: a reviewable artifact or trace showing that input schema was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

4. Output schema

Specify JSON or typed output fields, descriptions, enums, nullability, confidence representation, and examples of valid and invalid outputs.

- Required evidence: a reviewable artifact or trace showing that output schema was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Tool contract

Describe tool purpose, permissions, argument semantics, side effects, preconditions, and post-call validation. Separate read-only tools from write tools.

- Required evidence: a reviewable artifact or trace showing that tool contract was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

6. Failure behavior

Define when the model must abstain, request missing information, return a validation error, retry, or escalate. Prohibit silent guessing for required fields.

- Required evidence: a reviewable artifact or trace showing that failure behavior was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

7. Test design

Create happy-path, boundary, ambiguous, adversarial, multilingual, and malformed-input cases. Include expected structured outputs and allowed variation.

- Required evidence: a reviewable artifact or trace showing that test design was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

8. Change review

Compare candidate prompts against the current version using fixed model settings and statistically meaningful samples. Review both aggregate metrics and severe individual failures.

- Required evidence: a reviewable artifact or trace showing that change review was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

9. Release and monitoring

Version the prompt, schema, model, and evaluation set together. Monitor parsing failures, refusal anomalies, drift, and tool-call distributions.

- Required evidence: a reviewable artifact or trace showing that release and monitoring was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Entry Criteria

Work may enter the methodology when a named owner, defined user outcome, initial data classification, and target environment are available. Teams should also identify downstream consumers, irreversible actions, expected traffic, and the cost of wrong answers. If these inputs are unknown, the first deliverable is a discovery brief rather than an implementation.

6. Design Review Expectations

The design review should focus on concrete decisions and evidence. Reviewers inspect architecture diagrams, state or data schemas, model and tool dependencies, evaluation plans, privacy boundaries, fallback behavior, and operating metrics. Open issues must have owners and dates. A design is not approved merely because a model can demonstrate the happy path; the team must show how the system behaves with missing inputs, conflicting evidence, unavailable services, malformed outputs, and unauthorized requests.

7. Documentation and Traceability

Each release maintains a versioned record of requirements, decisions, prompts or policies, model identifiers, data or index versions, test results, known limitations, and approvers. Traceability should allow an incident responder to determine which configuration produced a specific output. Documentation must be concise enough to maintain but detailed enough for a new engineer to reproduce critical behavior.

8. Change Management

Changes are categorized as low, medium, or high impact. Low-impact changes include wording or monitoring adjustments with no behavior change. Medium-impact changes affect routing, prompts, thresholds, or non-sensitive data. High-impact changes include model replacement, new write capabilities, new sensitive data, changed jurisdictions, or altered decision authority. Medium and high-impact changes require targeted regression testing; high-impact changes repeat the relevant risk and approval reviews.

9. Operational Readiness

Before launch, the team confirms monitoring, alert ownership, on-call procedures, rollback, rate limits, access controls, data retention, incident communication, and user support. Synthetic probes should continuously test critical paths. Runbooks should describe both technical recovery and the user-facing behavior during degradation.

10. Continuous Improvement

Production evidence is reviewed on a regular cadence. Teams analyze failures by root cause rather than by surface symptom, distinguish model errors from retrieval, tool, data, policy, or interface errors, and prioritize improvements by user harm and frequency. Confirmed defects become regression tests. Metrics are segmented by use case, language, tenant, and risk tier to avoid hiding poor performance inside global averages.

12. Roles and Responsibilities

Role	Accountability
Prompt owner	Maintains intent and approves changes.
Application engineer	Implements schemas and validators.
Domain reviewer	Validates terminology and decisions.
Safety reviewer	Tests misuse and policy bypass.
Evaluation engineer	Maintains regression suites and scorecards.

13. Measurement Framework

Metric	Definition and Use
Schema validity	Outputs passing parser and type validation.
Instruction adherence	Cases satisfying explicit behavioral rules.
Task accuracy	Correct decisions or content against gold labels.
Abstention quality	Correct abstentions among cases requiring them.
Tool-call precision	Tool calls that are necessary and correctly formed.
Regression severity	Weighted count of new failures introduced by a change.
Prompt token cost	Input tokens attributable to reusable instructions.

Metrics must be segmented by use case and risk tier. Teams should define a baseline, target, alert threshold, and owner for each production metric. A metric without an action rule is considered observational rather than operational.

14. Worked Example

A support triage prompt must classify urgency and select one of five queues. The contract defines fields for category, severity, required evidence, and confidence. It prohibits inventing account details and requires escalation when safety-related language is present. A proposed wording change improves average accuracy but causes two safety cases to route incorrectly; the release is blocked despite the aggregate gain.

15. Risk Register and Controls

Risk	Primary Controls
Prompt injection	Delimit untrusted content and route authorization through code, not model obedience.
Example overfitting	Use diverse examples and hidden tests.
Schema brittleness	Allow optional explanatory fields while keeping action fields strict.
Model upgrade drift	Re-run the full contract suite for every model or parameter change.

Long prompts	Remove redundant prose and move deterministic rules into code.
Undocumented assumptions	Record all external dependencies and default behaviors in the contract.

16. Release Gate

- A named business and technical owner has approved the intended use and operating boundaries.
- Required architecture, security, privacy, and risk reviews are complete.
- Evaluation results meet minimum thresholds and no severe unresolved defect remains.
- Monitoring, alerting, rollback, and incident procedures have been tested.
- User-facing disclosures, approval checkpoints, and fallback behavior are implemented where required.
- All model, prompt, tool, data, index, and policy versions are recorded in the release manifest.

17. Review Checklist

- Is the user outcome specific, measurable, and appropriate for AI assistance?
- Are authority boundaries and prohibited actions explicit?
- Can every important output be traced to inputs, evidence, and configuration?
- Does the design handle missing, conflicting, stale, or malicious inputs?
- Are sensitive data, permissions, retention, and deletion controlled?
- Are severe failures represented in the evaluation suite?
- Can the service degrade or stop safely when dependencies fail?
- Are metrics connected to owners and response procedures?

18. Appendices

Prompt contract template

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Prompt Contract Engineering Standard in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Structured output checklist

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Prompt Contract Engineering Standard in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Regression review form

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Prompt Contract Engineering Standard in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Tool description template

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Prompt Contract Engineering Standard in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Revision History

Version	Date	Summary
1.0	21 July 2026	Initial concise methodology edition.
2.0	21 July 2026	Expanded edition with detailed lifecycle, controls, roles, metrics, examples, risks, release gates, and appendices.